Overview

This project implements a Multi-View 3D Reconstruction algorithm. The goal is to estimate the 3D world coordinates of an object captured by multiple cameras from different viewpoints. The EPFL Soldiers tracking dataset was used. Assuming known intrinsic and extrinsic camera parameters (the pinhole camera model), we infer 3D world points using a closed-form least-squares solution to minimize the projection error.

The Camera Projection Model

To begin, we define the Intrinsic Parameter Matrix Λ.

$$\Lambda = \begin{bmatrix} \phi_{xj} & \gamma_j & \delta_{xj} \\ 0 & \phi_{yj} & \delta_{yj} \\ 0 & 0 & 1 \end{bmatrix}$$

The model relates homogeneous image coordinates $[x_j, y_j, 1]^T$ from world coordinates $[u, v, w, 1]^T$ via intrinsic and extrinsic (rotation and translation) parameters, scaled by the factor λ_j.

$$\lambda_j \begin{bmatrix} x_j \\ y_j \\ 1 \end{bmatrix} = \Lambda \begin{bmatrix} \omega_{11j} & \omega_{12j} & \omega_{13j} & \tau_{xj} \\ \omega_{21j} & \omega_{22j} & \omega_{23j} & \tau_{yj} \\ \omega_{31j} & \omega_{32j} & \omega_{33j} & \tau_{zj} \end{bmatrix} \begin{bmatrix} u \\ v \\ w \\ 1 \end{bmatrix}$$

Our objective is to solve for the world coordinates. We multiply both sides by Λ^{-1}, which results in the normalized image coordinates x'_j and y'_j.

$$\begin{bmatrix} \lambda_j x'_j \\ \lambda_j y'_j \\ \lambda_j \end{bmatrix} = \begin{bmatrix} \omega_{11j} & \omega_{12j} & \omega_{13j} & \tau_{xj} \\ \omega_{21j} & \omega_{22j} & \omega_{23j} & \tau_{yj} \\ \omega_{31j} & \omega_{32j} & \omega_{33j} & \tau_{zj} \end{bmatrix} \begin{bmatrix} u \\ v \\ w \\ 1 \end{bmatrix},$$

To handle the homogeneous coordinate scaling factor λ_j, we substitute λ_j with the last row of the matrix multiplication on the right side of the equation, cancelling out the last row for both equations.

$$\lambda_j = \omega_{31j} u + \omega_{32j} v + \omega_{33j} w + \tau_{zj}$$

$$\begin{bmatrix} (\omega_{31j}u + \omega_{32j}v + \omega_{33j}w + \tau_{zj})x'_j \\ (\omega_{31j}u + \omega_{32j}v + \omega_{33j}w + \tau_{zj})y'_j \end{bmatrix} = \begin{bmatrix} \omega_{11j} & \omega_{12j} & \omega_{13j} & \tau_{xj} \\ \omega_{21j} & \omega_{22j} & \omega_{23j} & \tau_{yj} \end{bmatrix} \begin{bmatrix} u \\ v \\ w \\ 1 \end{bmatrix}$$

In this form, we can isolate τ to one side of the equation.

$$\begin{bmatrix} \omega_{31j}x'_j - \omega_{11j} & \omega_{32j}x'_j - \omega_{12j} & \omega_{33j}x'_j - \omega_{13j} \\ \omega_{31j}y'_j - \omega_{21j} & \omega_{32j}y'_j - \omega_{22j} & \omega_{33j}y'_j - \omega_{23j} \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} \tau_{xj} - \tau_{zj}x'_j \\ \tau_{yj} - \tau_{zj}y'_j \end{bmatrix}$$

Now we can replace each element of the matrix with their respective $a_j, b_j, c_j, d_j, e_j,$ and f_j for cleaner notation.

$$\begin{bmatrix} a_j & b_j & c_j \\ d_j & e_j & f_j \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} g_j \\ h_j \end{bmatrix}$$

By stacking J cameras, the equation can support information from multiple perspectives.

$$\begin{bmatrix} a_1 & b_1 & c_1 \\ d_1 & e_1 & f_1 \\ a_2 & b_2 & c_2 \\ d_2 & e_2 & f_2 \\ \vdots & \vdots & \vdots \\ a_J & b_J & c_J \\ d_J & e_J & f_J \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} g_1 \\ h_1 \\ g_2 \\ h_2 \\ \vdots \\ g_J \\ h_J \end{bmatrix},$$

Furthermore, we substitute those matrices with the following notation for even cleaner notation.

$$A\mathbf{w} = \mathbf{b},$$

Finally, using Moore-Penrose Pseudoinverse $A^\dagger$ isolates the world coordinates $\hat{\mathbf{w}}$.

$$\hat{\mathbf{w}} = A^\dagger \mathbf{b},$$

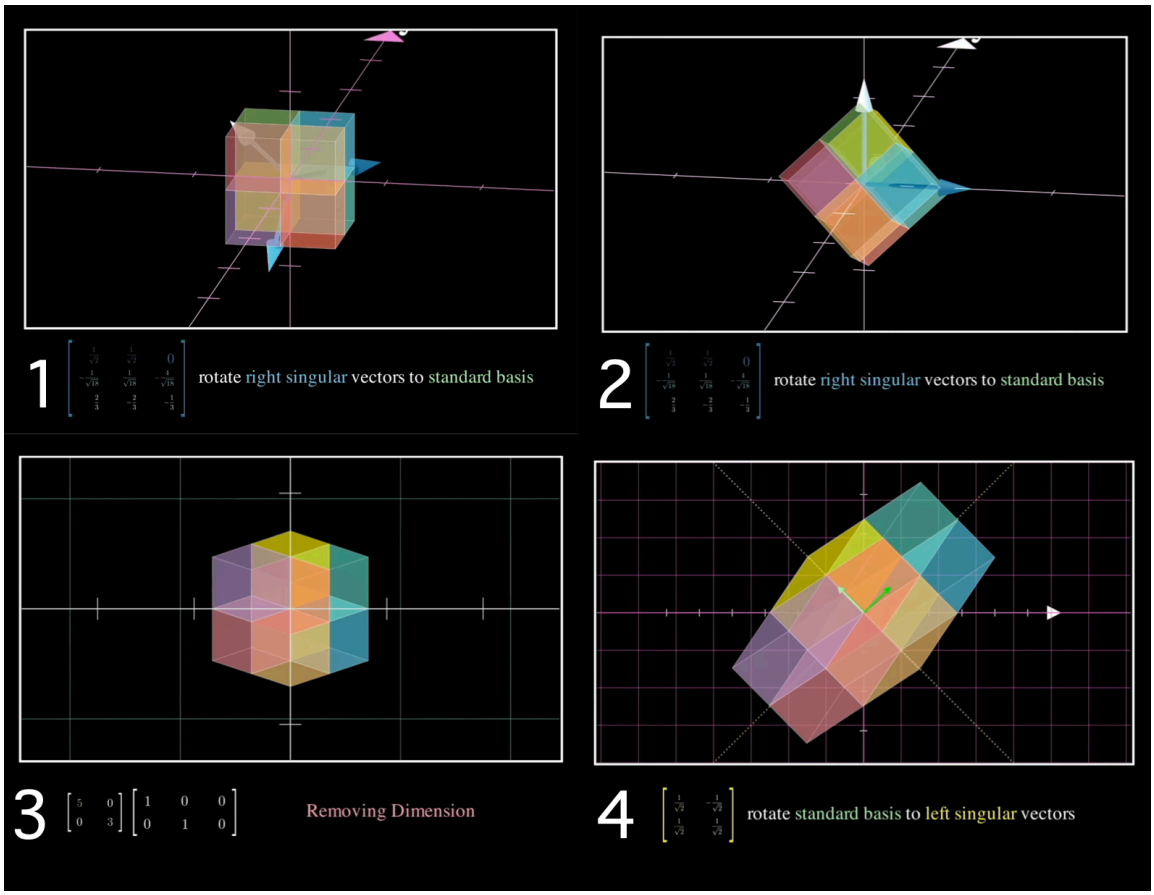
The math was adapted from Dr. Ribeiro's notes (FIT) and equations 15.16, 15.17, 15.34 from Computer Vision: Models, Learning, and Inference by Simon J.D. Prince.

Pseudo Matrices

A core component of Pseudo Matrices is Single Value decompostion (SVD). Its purpose is dividing a linear transformation into multiple components each with its own attributes:

$$M = U\Sigma V^T$$

Where V^T is in charge of rotating or tilting to orthogonal unit vectors (change of basis), the Σ is a rectangular transformation matrix (e.g transfroming a 3D shape to a 2D) and U represents the original directions before the V^T transformation in respects of Σ. The numbered graphics below are from this video and V^T is 2, Σ is 3, and U is 4.



Pseudo Matrices uses variation of:

$$A^\dagger = V\Sigma^T U^T$$

The biggest difference is that we are transposing the Σ. Due to change of dimensions the orthogonal matrices (U and V) had to change order because at its essence what a rectangular transposed matrix is grabbing a 3D space and squishing it into a single line. This is useful because how the 3D world points estimator works is using a single line that goes through the x and y coordinates in an image to estimate the w coordinates.

Implementation

In [1]:
```
import os
import imageio
import json
import numpy as np
```

```
import cv2
import matplotlib.pyplot as plt
import gdown
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

In [91]:

```
#Extracts files from json
def json_read(filename):
  try:
      with open(os.path.abspath(filename)) as f:
          data = json.load(f)
      return data
  except:
      raise ValueError("Unable to read JSON {}".format(filename))


#Extracts the camera parameters and returns np arrays as
#intrinsic(Lambda)/(N, 3, 3), rotation(Omega)/(N, 3, 3),
# translation(tau)/(N, 3, 1), with N being amount of cameras.
def all_camera_parameters(cameras, pathname):

  #Reads the json file containing the calibration data
  calibration = json_read("/{}".format(pathname) + "/calibration.json")

  #Declares future stacked arrays
  Lambda_stacked, Omega_stacked, tau_stacked = [],[],[]

  for view in cameras:

  #Gets the calibration matrices from each cameras
    Lambda_stacked.append(np.array(calibration[view]['K']))     #Intrinsic p
    Omega_stacked.append(np.array(calibration[view]['R']))      #Rotation

    #Addresses nuances of different matrices implementation
    #(ex (3,1) vs (3,))
    tau_stacked.append(np.array(calibration[view]['t']).reshape(3,1))      #

  #Returns the stacked arrays as np arrays
  return np.array(Lambda_stacked), np.array(Omega_stacked), np.array(tau_sta


#Estimates the points in 3D space
#Required dimensions for np arrays, with N being the amount of cameras and P
#xy = (N, P >= 2, 2) (Lambda = (N, 3, 3)
#Omega = (N, 3, 3), tau = (N, 3, 1)
def estimated3Dpoints(xy, Lambda, Omega, tau):

  #Converts into homogeneous coordinates, (6, 4, 2) -> (6, 4, 3)
  column_ones = np.ones((xy.shape[0], xy.shape[1], 1))
  XY_homog = np.concatenate([xy, column_ones], axis=2)

  #Matrix mulitplication of xy with inverse of lambda
  #= (6, 4, 3) = (6, 4, 3) @ (6, 3, 3)
  xy_norm_homog = XY_homog @ np.linalg.inv(Lambda).transpose(0, 2, 1)

  #Normalizes homogeneous coordinate by dividing the z axis in xy_norm_homog
  #to cancel it out
```

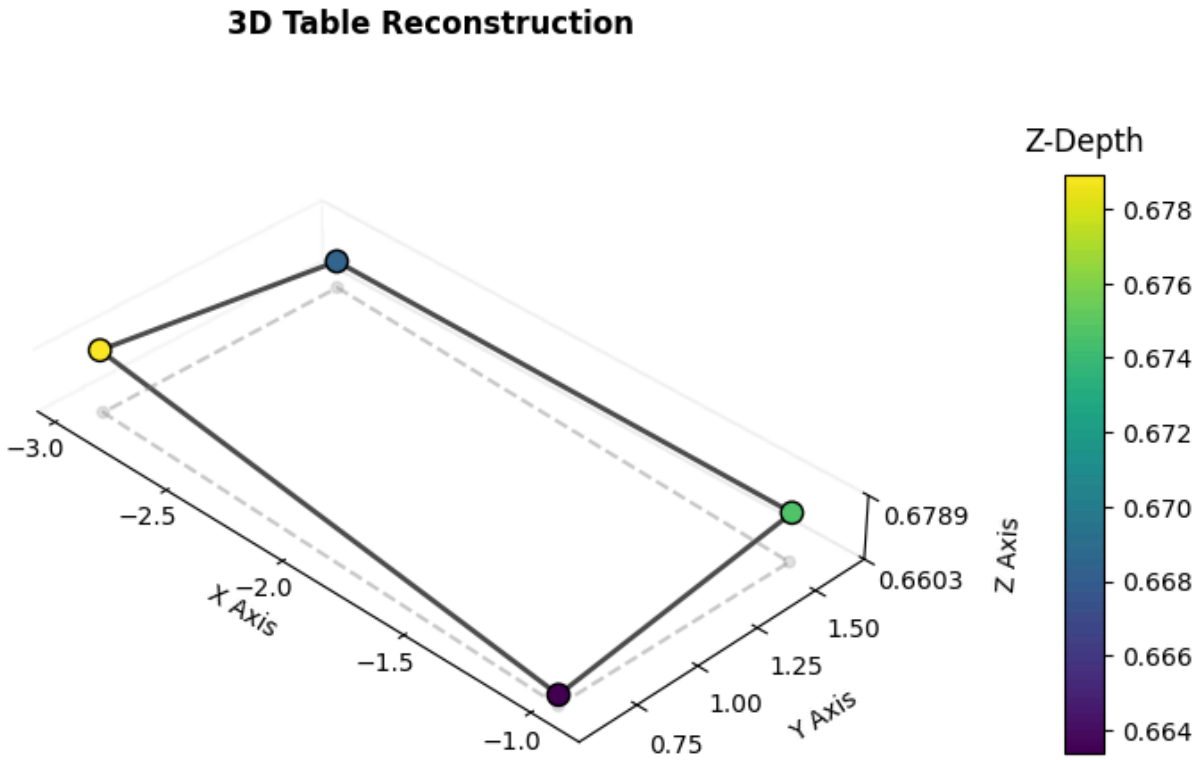
```
  xy_norm = xy_norm_homog[:, :, :2] / xy_norm_homog[:,:,2:3]

  #Converts back into cartesian coordinates shape (6, 4, 2, 1)
  xy_norm = xy_norm.reshape(XY_homog.shape[0],XY_homog.shape[1], 2, 1)

  #We need match Omega's dimension by adding new axis to construct Matrix A
  #Omega[all cameras, all points, both x y coodinates,
  # last row, all columns] = (6, 1, 1, 3)
  A = xy_norm * Omega[:, np.newaxis, np.newaxis, 2, :]

  #Rearranging to isolate tau to finalize matrix A
  A = A - Omega[:, np.newaxis, :2, :]

  #Constructs Matrix B
  #tau[all cameras, all points, third row, every column]
  # - tau[all cameras, all points, third to fourth row, every column]
  B = tau[:,np.newaxis, :2,:] - (xy_norm*tau[:, np.newaxis, 2:3,:])

  #We swap out cameras and points columns to align with the math.
  #We flatten out cameras and equation vectors (what became of
  #the extrinsic and intrinsic parameters) because each point requires
  #its own system of equation.
  A_final = A.transpose(1, 0, 2, 3).reshape(xy.shape[1], xy.shape[0] * 2, 3)
  B_final = B.transpose(1, 0, 2, 3).reshape(xy.shape[1], xy.shape[0] * 2, 1)

  #We compute the Pseudo inverse
  w = np.linalg.pinv(A_final) @ B_final

  #Returns shape (4,3,1)
  return w

#Does a 3D plot of the object
def plotReconstructedObject(w_hat, lines = True):

    #Extracts the four points coordinates
    x = w_hat[:, 0]
    y = w_hat[:, 1]
    z = w_hat[:, 2]

    #Makes the figure 9 inches wide and 7 inches tall
    fig = plt.figure(figsize=(9, 7))

    #Prepares the figure to display 3D points
    ax = fig.add_subplot(projection='3d')

    #Creates scatter plots points using: viridis color map, size of 80 (s),
    #and black borders.
    sc = ax.scatter(x, y, z, c=z, cmap='viridis', marker='o', s=80, alpha=1.

    if lines:
      #Creates the lines in between in between each point. Appending the fir
      #last point and the first
      ax.plot(np.append(x, x[0]), np.append(y, y[0]), np.append(z, z[0]), c=

    #Adds Z-depth color bar, shrink controls how tall the bar is, aspect the
    #pad is space around the bar
```

```
    fig.colorbar(sc, ax=ax, shrink=0.6, aspect=15, pad=0.17).ax.set_title('Z

    #Calculates shadow for the z axis by grabbing the minimum z coordinate
    #and subtracting it by 20% of total height of the z coordinates
    z_floor = np.min(z) - np.ptp(z) * 0.2

    #Plots the shadow, replaces z axis with the floor
    ax.scatter(x, y, np.full_like(z, z_floor), c='gray', marker='o', alpha=0
    if lines:
      ax.plot(np.append(x, x[0]), np.append(y, y[0]), np.full_like(np.appenc

    #Scales plot according to the data
    ax.set_zlim(z_floor, np.max(z))
    #Added failsafe to the z axis in case z values are too small
    ax.set_box_aspect((np.ptp(x), np.ptp(y), max(np.ptp(z), np.ptp(x)*0.1)))

    #Controls the tick marks in the graph
    ax.locator_params(axis='x', nbins=5)
    ax.locator_params(axis='y', nbins=5)
    ax.locator_params(axis='z', nbins=2)

    #Forces 2 ticks for z axis
    ax.set_zticks([z_floor, np.max(z)])

    #Removes grid
    ax.xaxis.pane.fill = False
    ax.yaxis.pane.fill = False
    ax.zaxis.pane.fill = False
    ax.grid(False)

    #Labels
    ax.set_xlabel('X Axis', labelpad=10)
    ax.set_ylabel('Y Axis', labelpad=8)
    ax.set_zlabel('Z Axis', labelpad=8)
    ax.set_title('3D Table Reconstruction', weight='bold', pad=1)

    #Different perspective
    ax.view_init(elev=35, azim=-45)

    plt.show()

#Calculates MSE by estimating XY from estimated world coordinates
def reprojectionError(xy, Lambda, Omega, tau, w_est):

  #We need to add extra one to match our equation
  column_ones = np.ones((w_est.shape[0], 1, w_est.shape[2]))
  w_est_homogenous = np.concatenate([w_est, column_ones], axis=1)

  #Constructs matrix for our equation
  ome_tau = np.concatenate([Omega, tau], axis = 2)

  #Replicates the formula
  xy_homog = Lambda[:, np.newaxis, :, :] @ ome_tau[:, np.newaxis, :, :] @ w_

  #Normalizes homohenours coodinates
  xy_homog = xy_homog / xy_homog[:,:, 2:3, :]
```

```
  #Erases the extra one now and grabs the first two coordinates
  xy_est = xy_homog[:,:, :2, 0]

  return np.mean((xy-xy_est)**2)
```

In [94]:

```
filename= 'soldiers_bc_epfl.tar.gz'
pathname = 'content'
url = 'https://drive.google.com/uc?id=17rwFmew4XoQOoDDGs7yO2ovBmMZ5Qcf7'
cameras = ['3_2', '3_4', '6_1', '6_3', '6_4', '6_5']
download = True # TURN OFF IN REPEAT RUNS
points = 4

if download:
  #Gets the camera information
  print(gdown.download(url, "/{}/".format(pathname), quiet = True))
  #Extracts the contents of file .tar
  os.system('tar -xvf ' + "/{}/{} ".format(pathname,filename) + '-C /{}/'.fo

#XY coordinates need to be in the same order as the cameras,
#and the points gotta match. Ex: [1196,320], [816, 306]
#are the same corner of the table, in different cameras persepctives
XY = np.array([
    [1197, 323], [1273, 320], [1183, 296], [1115, 296],

    [818, 308], [794, 322], [969, 338], [979, 321],

    [775, 318], [824, 318], [819, 226], [771, 225],

    [908, 264], [933, 233], [742, 225], [708, 251],

    [1029, 375], [1021, 344], [812, 344], [808, 374],

    [832, 344], [876, 364], [1028, 351], [973, 332]
])

#XY(cameras, points, axis per point)
XY = XY.reshape(len(cameras), points, 2)

#Extracts camera parameters from json file
Lambda, Omega, Tau = all_camera_parameters(cameras, pathname)

#Estimates world coordinates based on image coordinates
allwestimated = estimated3Dpoints(XY, Lambda, Omega, Tau)

#Plots reconstructed object
plotReconstructedObject(np.array(allwestimated))

#Calculates the reprojection error
print("\n" + "Reprojection Error: " + str(reprojectionError(XY, Lambda, Omeg
```




```
Reprojection Error: 8.234676419445417
```

In my first attempt at this task, my reprojection error was 2,315,931,076,561.4. Initially, I thought this was due to scaling, but upon further inspection, there were many issues.

First, the way I implemented the math estimated the four points per camera perspective and averaged the rest of the camera perspectives. The mistake becomes clear once you shift your perspective of matrices being spatial transformations in space to a system of equations. From the system of equations point of view, using stacked matrices allows the model to account for all the camera perspectives simultaneously.

Second, we needed to transpose the image coordinates to match the derived math. Third, the MSE was squared twice. Lastly, fixing human mislabeled points and using a more precise tool (macOS preview / resolution info vs OpenCV) got us to our final 8.23 error. Square rooting it leaves us with a 2.86 pixel error, which is outstanding! Plotting the estimated points, the figure resembles the shape of the table. Even the lengths of the edges seem appropriate. The image below was me recording the pixel coordinates with macOS preview before I switched to OpenCV.

3_2

0 [1194, 320]

1 [1274, 318]

2 [1180, 296]

3 [1112, 295]